



BIG DATA LAB 2

Eng. Heba Abughali



2026

LAB 2: BIG DATA STORAGE & FILE SYSTEMS

WEEK 2: BIG DATA STORAGE AND PROCESSING

Objectives

Understanding the difference between traditional and distributed file systems

Using Google Cloud Storage as an example of distributed storage

Experiencing the storage and reading of large files

Lab Activities

Part 1: File format

Choosing the right file format is often a trade-off between human readability, storage efficiency, and processing speed.

Feature	CSV	JSON	Parquet
Type	Row-based	Hierarchical / Document	Columnar
Readability	Human-readable	Human-readable	Binary (Machine-only)
Schema	Flat (No metadata)	Nested (Self-describing)	Strongly Typed
Compression	Low	Low	High
Speed	Slow for large data	Very slow	Very Fast

Part 2: Simulating Distributed Storage

Replication is the core mechanism used to ensure Reliability and Availability in Big Data ecosystems.

- Replication Factor (3x): By default, HDFS splits data into blocks and stores three identical copies of each block across different nodes.
- Storage Overhead: High reliability comes at a cost. A replication factor of 3 means you need 3x the storage capacity (e.g., 100 MB of data requires 300 MB of disk space).
- Fault Tolerance: This setup ensures the system can survive up to two simultaneous node failures without any data loss, as at least one copy remains accessible.

Part 3: Column-Oriented vs Row-Oriented Storage

Feature	Row-Oriented (e.g., CSV, SQL)	Column-Oriented (e.g., Parquet)
Data Layout	Stores data row by row: (ID, Name, Date), (ID, Name, Date)	Stores data column by column: (IDs), (Names), (Dates)
Best Use Case	OLTP (Online Transactional Processing)	OLAP (Online Analytical Processing)
Ideal Task	Adding/updating a single record (e.g., User Profile)	Aggregating large data (e.g., Calculating Average Price)
Read Efficiency	Reads the entire row, even if you only need one column	Reads only the specific columns requested
Compression	Low (data types vary within a row)	Very High (same data types are grouped together)

LAB EXERCISE

Exercise 1: File Format Comparison

Create a dataset with 2 million rows

Test these formats: (CSV, JSON, Parquet, Feather, Pickle) , then write a comparison between these formats in terms of their use, features, and drawbacks.

Measure:

1. Write time

2. Read time

3. File size

4. Query performance (filter + aggregation) , i.e., `SELECT AVG(value1) FROM table_name WHERE category = 'A'`

Where

```
df = pd.DataFrame({
    'id': range(n_rows),
    'timestamp': pd.date_range('2023-01-01', periods=n_rows, freq='1s'),
    'category': np.random.choice(['A', 'B', 'C', 'D', 'E'], n_rows),
    'value1': np.random.randn(n_rows),
    'value2': np.random.randn(n_rows),
    'value3': np.random.uniform(0, 100, n_rows),
    'text': [f'Transaction_{i}' for i in range(n_rows)]
})
```

Result example:

Format	Write Time (s)	Read Time (s)	Query Time (s)	File Size (MB)	Compression Ratio
CSV	12.483673	4.891224	0.189070	202.814571	1.480671
JSON	7.314761	9.973162	0.162457	288.878354	1.039543
Parquet	0.859839	1.505852	0.172378	78.361026	3.832282
Feather	0.649016	1.026009	0.148305	90.113070	3.332497
Pickle	0.488968	0.214455	0.162808	121.025637	2.481305

Exercise 2: Partition Strategy

Given a sales dataset with 'date' column

Implement partition by:

1. Date (daily partitions)

2. Region (geographic partitions)

3. Product category

Compare:

- Number of partitions

- Partition size distribution

- Query performance for different scenarios

Exercise 3: Big Data Storage & Cost Planning

Create a Python tool to calculate storage requirements, network replication time, and costs for 10TB of data. Additionally, visualize how different compression ratios impact the total storage and budget.

Tasks:

- Storage Analysis: Calculate the final size after Compression and 3x Replication.
- Network Analysis: Estimate the time needed to replicate data at 100 Mbps.
- Costing: Estimate AWS S3 (\$0.023/GB) monthly/yearly bills.
- Visualization: Plot two graphs: where Compression Ratio (from 1x to 5x).
 - Storage vs. Compression Ratio.
 - Yearly Cost vs. Compression Ratio.
- Test Scenario: * Raw Data: 10,000 GB | Replication: 3x | Compression: 4x

Part 1: Local vs Distributed Storage

```
import pandas as pd
import numpy as np
import time
from google.colab import files
import io

# Generate a large dataset
print("Generating 5 million rows...")
large_df = pd.DataFrame({
    'id': range(5000000),
    'value': np.random.randn(5000000),
    'category': np.random.choice(['A', 'B', 'C', 'D'], 5000000),
    'timestamp': pd.date_range('2024-01-01', periods=5000000, freq='1s')
})

print(f"Dataset size in memory: {large_df.memory_usage().sum() / 1024**2:.2f} MB")

# Save in different formats
formats = ['csv', 'parquet', 'json']
save_times = {}
file_sizes = {}

for fmt in formats:
    print(f"\n--- Testing {fmt.upper()} format ---")

    # Save
    start = time.time()
    if fmt == 'csv':
        large_df.to_csv(f'data.{fmt}', index=False)
    elif fmt == 'parquet':
        large_df.to_parquet(f'data.{fmt}', index=False)
    elif fmt == 'json':
        large_df.to_json(f'data.{fmt}', orient='records', lines=True)
    save_time = time.time() - start
    save_times[fmt] = save_time

    # Get file size
    import os
    file_size = os.path.getsize(f'data.{fmt}') / 1024**2
    file_sizes[fmt] = file_size

    print(f"Save time: {save_time:.2f} seconds")
    print(f"File size: {file_size:.2f} MB")
    print(f"Compression ratio: {large_df.memory_usage().sum() / 1024**2 / file_size:.2f}x")

plt.tight_layout()
plt.show()
```

```

# Visualization
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.bar(formats, [save_times[f] for f in formats])
ax1.set_ylabel('Time (seconds)')
ax1.set_title('Save Time by Format')
ax1.grid(axis='y')

ax2.bar(formats, [file_sizes[f] for f in formats])
ax2.set_ylabel('Size (MB)')
ax2.set_title('File Size by Format')
ax2.grid(axis='y')

print("\n🔑 Key Insights:")
print("• Parquet is optimized for big data (columnar storage)")
print("• CSV is human-readable but slow and large")
print("• JSON is flexible but inefficient for analytics")

```

Part 2: Simulating Distributed Storage

```
# Simulate data partitioning (like HDFS)
print("=== SIMULATING DISTRIBUTED FILE SYSTEM ===\n")

def partition_data(df, n_partitions):
    """Simulate splitting data across multiple nodes"""
    partition_size = len(df) // n_partitions
    partitions = []

    for i in range(n_partitions):
        start_idx = i * partition_size
        end_idx = start_idx + partition_size if i < n_partitions - 1 else len(df)
        partition = df.iloc[start_idx:end_idx].copy()
        partitions.append(partition)

    # Save partition
    partition.to_parquet(f'partition_{i}.parquet', index=False)
    size = os.path.getsize(f'partition_{i}.parquet') / 1024**2
    print(f"Partition {i}: {len(partition):,} rows, {size:.2f} MB")

    return partitions

# Create partitions (simulating HDFS blocks)
n_partitions = 10
partitions = partition_data(large_df, n_partitions)

print(f"\nTotal partitions: {n_partitions}")
print(f"Rows per partition: ~{len(large_df) // n_partitions:,}")

# Simulate reading from distributed storage
print("\n=== READING FROM DISTRIBUTED STORAGE ===")
start = time.time()
reconstructed_df = pd.concat([pd.read_parquet(f'partition_{i}.parquet')
                             for i in range(n_partitions)],
                             ignore_index=True)
read_time = time.time() - start

print(f"Read time: {read_time:.2f} seconds")
print(f"Rows read: {len(reconstructed_df):,}")
print(f>Data integrity: {'✓ PASSED' if len(reconstructed_df) == len(large_df) else '✗ FAILED'})

# Simulate replication (HDFS default: 3 replicas)
print("\n=== SIMULATING REPLICATION ===")
replication_factor = 3
total_storage = sum(file_sizes.values())
replicated_storage = total_storage * replication_factor

print(f"Original storage: {total_storage:.2f} MB")
print(f"With {replication_factor}x replication: {replicated_storage:.2f} MB")
print(f"Storage overhead: {(replication_factor - 1) * 100:.0f}%")
print(f"Benefit: Fault tolerance - data survives {replication_factor - 1} node failures")
```

Part 3: Column-Oriented vs Row-Oriented Storage

```
print("=== COLUMN-ORIENTED VS ROW-ORIENTED STORAGE ===\n")

# Create sample data
data = pd.DataFrame({
    'customer_id': range(1000000),
    'name': ['Customer_' + str(i) for i in range(1000000)],
    'age': np.random.randint(18, 80, 1000000),
    'income': np.random.randint(20000, 200000, 1000000),
    'city': np.random.choice(['Cairo', 'Alex', 'Giza', 'Luxor'], 1000000)
})

# Row-oriented (CSV) - reads entire rows
print("Scenario: SELECT age, income FROM customers WHERE age > 30")
print("\n--- Row-Oriented (CSV) ---")
start = time.time()
data.to_csv('row_oriented.csv', index=False)
df_csv = pd.read_csv('row_oriented.csv')
result_csv = df_csv[df_csv['age'] > 30][['age', 'income']]
csv_time = time.time() - start
print(f"Time: {csv_time:.3f} seconds")
print(f"Read ALL columns, then filtered")
print(f"File size: {os.path.getsize('row_oriented.csv')} / 1024**2:.2f MB")

# Column-oriented (Parquet) - reads only needed columns
print("\n--- Column-Oriented (Parquet) ---")
start = time.time()
data.to_parquet('column_oriented.parquet', index=False)
df_parquet = pd.read_parquet('column_oriented.parquet',
                             columns=['age', 'income'])
result_parquet = df_parquet[df_parquet['age'] > 30]
parquet_time = time.time() - start
print(f"Time: {parquet_time:.3f} seconds")
print(f"Read ONLY age & income columns")
print(f"File size: {os.path.getsize('column_oriented.parquet')} / 1024**2:.2f MB")

print(f"\n📊 Speedup: {csv_time / parquet_time:.2f}x faster")
print(f"📦 Compression: {os.path.getsize('row_oriented.csv')} /  
os.path.getsize('column_oriented.parquet'):.2f}x smaller")

print("\n💡 Why Parquet is better for Big Data:")
print("✓ Reads only needed columns (columnar storage)")
print("✓ Better compression (similar values grouped)")
print("✓ Efficient for analytical queries")
print("✓ Used by Spark, Hive, Impala")
```